

Attention Over Terrain Profiles

Learning To Approximate ITM With A Transformer

- 15-20 minute talk
- Background: ITM, US elevation data, legacy implementation
- Main point: concept validation, not speedup

Alex Punnen

Talk Roadmap

1. What ITM is and why people still use it
2. US elevation data: NED and the modern 3DEP context
3. Why an attention model is a reasonable idea here
4. Dataset, model, training, and results
5. What worked, what did not, and what remains open

What Is ITM?

- ITM = Irregular Terrain Model, also known as Longley-Rice
- Developed at the Institute for Telecommunication Sciences (ITS / NTIA)
- Designed for radio propagation prediction from 20 MHz to 20 GHz
- Uses frequency, distance, antenna heights, and terrain profile
- Outputs median transmission loss plus variability terms

Why it matters:

- It is still a practical engineering baseline
- It is physics-based and widely trusted
- It remains useful for planning and interference studies

Why Terrain Matters

- Two links with the same distance can have very different loss
- The terrain profile controls:
 - line-of-sight blockage
 - diffraction over ridges
 - beyond-horizon behavior
- In point-to-point mode, ITM consumes a 1D elevation profile along the path

Key observation:

- This profile is naturally a sequence
- The effect of one obstacle depends on where it sits relative to the transmitter and receiver

US Elevation Data: NED And 3DEP

- Historically, USGS distributed the National Elevation Dataset (NED)
- NED provided seamless raster elevation for CONUS, Alaska, Hawaii, and territories
- NED unified source DEMs into common datums, units, and grid structure
- In 2015, USGS retired the NED name under the broader 3D Elevation Program (3DEP)
- NED-era products are still widely used and cited

Why it matters here:

- ITM needs terrain profiles
- Terrain profiles come from sampling DEM/elevation rasters along transmitter-receiver paths

Legacy Implementation Matters

- Longley-Rice started in an older scientific-computing ecosystem, including Fortran-era implementations
- Over time, it was reimplemented and wrapped in C and C++
- This repo uses a compiled Python-accessible shared object:
 - `itm/itm_its.cpython-310-x86_64-linux-gnu.so`
- That means the baseline is not a toy implementation

Takeaway:

- If the learned model is slower today, that is against a mature compiled baseline
- That makes the comparison meaningful

Why Try Attention At All?

- Terrain is a variable-length ordered sequence
- Important interactions can be far apart in the profile
- Relevance is context-dependent:
 - the same hill matters differently at different frequencies
 - transmitter and receiver heights change what is important
- Attention can model long-range interactions directly

Hypothesis:

- A sequence model with self-attention and cross-attention can learn the ITM mapping

Problem Formulation

- Input 1: elevation profile sequence
- Input 2: scalar link features
 - distance
 - center frequency
 - receiver height
 - access point height
- Output: path loss in dB

This is framed as sequence-to-scalar regression.

The goal of this project is:

- first, to test whether the mapping is learnable
- not yet to claim a production-speed replacement for ITM

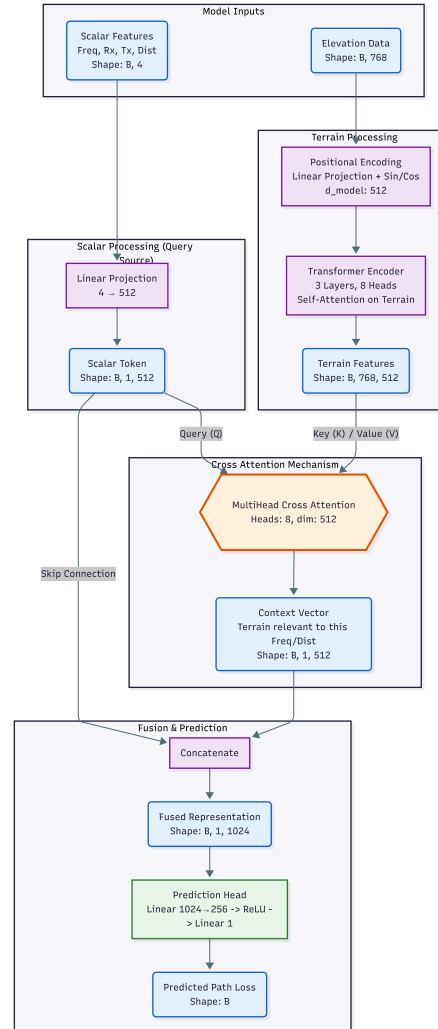
Dataset

- Source: ITM-generated supervision over US terrain profiles
- Public dataset:
 - https://huggingface.co/datasets/alexcpn/longely_rice_model
- About 7.8M samples
- 6 GHz band
- Distance range: 1.3 to 200 km
- TX height: 1.5 to 110 m
- RX height: 1.5 to 601 m
- Profile length: 47 to 766 points

Important practical lesson:

- Correct data alignment and normalization mattered a lot more than cosmetic architecture tweaks

Model Architecture



- Elevation sequence -> learned embedding + positional encoding

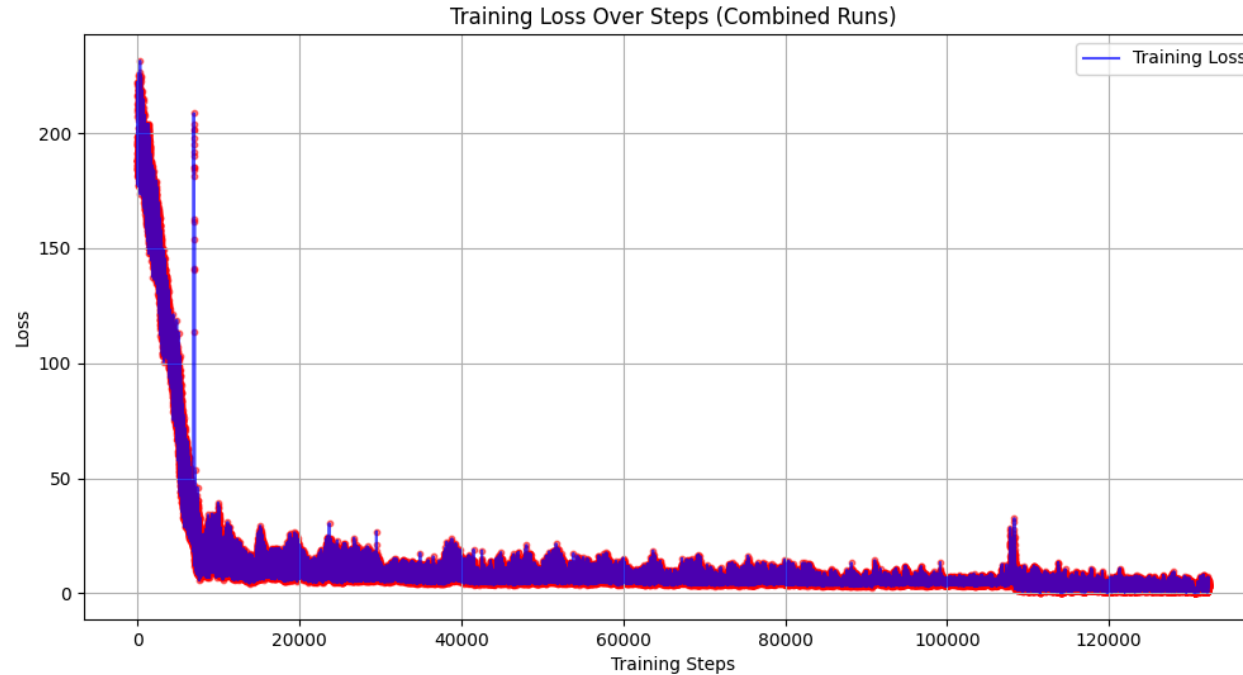
Training Setup

- Model dimension: 512
- 3 transformer encoder layers
- 8 attention heads
- Smooth L1 loss
- AdamW optimizer
- Gradient clipping
- One full pass over 7.8M+ samples

Normalization was critical:

- elevation normalized per sample
- scalar features normalized to stable ranges
- target path loss normalized for training stability

Did The Loss Actually Go Down?



- Yes, sharply at the start
- Loss dropped from roughly 230 to around 10 very early
- Then it plateaued with variance

Interpretation:

Accuracy Results

Metric	Value
RMSE	17.85 dB
MAE	10.94 dB
Median Error	5.00 dB
90th Percentile Error	31.02 dB
95th Percentile Error	41.59 dB

Improvement over iterations:

Version	RMSE
Baseline, no normalization	62.02 dB
+ input / target normalization	42.62 dB
+ dataset correction and full training	17.85 dB

What Does This Validate?

- The ITM mapping is learnable from terrain profiles plus link parameters
- Attention is a reasonable mechanism for this problem
- Training loss decreases materially with the right pipeline
- Data quality is a first-order issue
- Median error of 5 dB is already meaningful for a concept-validation result

This is the core claim of the work.

Runtime Reality Check

Current benchmark on this workstation:

Engine	Time / sample	Throughput
Direct ITM	11.0 us	91,082 pred/s
Transformer	1314.8 us	761 pred/s

Important nuance:

- The timed transformer benchmark excludes CPU -> GPU transfer
- A separate breakdown showed copy time is negligible here
- The slowdown is in the current forward path, not PCIe transfer

But:

- We have not yet isolated whether this is a fundamental architectural cost
- Or an engineering problem in the current implementation

Honest Interpretation

What we can say truthfully:

- The attention-based surrogate works as a learning experiment
- It does not yet beat the compiled ITM implementation on runtime
- The root cause of the runtime gap is still unresolved

What we should not say yet:

- "Transformers are inherently too slow for this task"
- "The runtime gap is definitely due to one specific bottleneck"
- "This is already a deployable replacement for ITM"

What Comes Next

First priority:

- lower loss and error further
- better scheduling
- longer training
- attention analysis
- improved architecture choices

Second priority:

- diagnose runtime properly
- profile kernels and masking behavior
- test lighter models
- test better inference stacks
- then revisit the speed question

Resources

- Repo:
 - https://github.com/alexcpn/elevation_transformer
- Dataset:
 - https://huggingface.co/datasets/alexcpn/longely_rice_model
- Weights:
 - https://huggingface.co/alexcpn/elevation_transformer/tree/main
- Benchmarks:
 - https://huggingface.co/alexcpn/elevation_transformer/tree/main/eval

Primary references:

- NTIA / ITS Longley-Rice documentation
- USGS NED documentation
- USGS 3DEP elevation products

Backup: Key Takeaway

Attention over 1D terrain profiles is a viable way to learn an ITM-like mapping.

The project already validates the concept.

The remaining open questions are:

- how much lower can the error go?
- what exactly is causing the runtime gap?
- can we turn this from a research surrogate into a practical one?